C/C++ Tutorial Exercises

Fitsum Assamnew
School of Electrical and Computer Engineering
Addis Ababa Institute of Technology
Addis Ababa University

November 24, 2022

In this tutorial, exercises that are used to recollect constructs of the C/C++ programming language are included. After thoroughly implementing the exercises included in this tutorial, students will be able to write their own programs.

This exercises assume you have some knowledge of the C/C++ language.

1 Multi-file Projects

In this exercise you will learn to develop multi-file projects.

- Create main.cpp, first.cpp, second.cpp, third.cpp and their corresponding header files. Add corresponding functions that start #Function (such as firstFunction) in each file other than the main.cpp. Each of the functions you created should print to the screen "I printing to the screen from #.cpp file" (where # should be replaced with the name of the file) on a new line.
- Add a function that adds two numbers and returns their result in first.cpp.
- Add a function that multiplies two numbers and returns their result in second.cpp
- Add a function that divides two numbers and returns their result in third.cpp
- In the main function, accept two numbers from the user and print out the results of the functions you created above to the screen.

*The arithmetic functions should work for integer and floating point numbers.

2 Structures and Arrays

Now that you have exercised with multiple file projects, let us progress to storing data in the computer. Structures are used to store data that have different data types on the other hand Arrays stored data that has similar data type. In the following exercises you will learn to store data in structures and multi-dimensional arrays.

- In an new project, create main.cpp, triangle.cpp, vector.cpp and matrix.cpp files and their corresponding header files.
- In traingle.cpp, create a structure that will hold information about a triangle. Also, in this file you should have functions that give information on a given triangle as the area of a triangle.

- In vector.cpp, define functions that add two vectors and multiply two vectors. Also, add a function accepts a vector of triangles and prints the area of each triangle on the screen.
- In matrix.cpp, add functions that accept two matrices then add or multiply them.
- Write functions that will populate the vectors and matrices with random data after the user gives the size of the matrix or vector. In this exercise, you should use dynamic allocation of memory for the vectors and matrices.

The functions defined for vectors and matrices should work for integer and floating point data types.

3 Measuring Runtime

Measuring the run-times of programs/functions is important to know the effect of an optimization or an algorithm that you implemented. There are multiple ways of measuring runtime for specific functions, however the accuracy of the measuring techniques varies. Also, the measured values for multiple runs of the same function/procedure can differ greatly as a computer is a multitasking environment. Hence, in order to have reliable measured value many measurements have to be collected and their average computed.

- Measure the vector addition and multiplication run-times using the wall-clock method (search for "wall clock timing measurement C++" on the internet)
- Now run the above experiment 10, 20, 30, 40, 50, 100, 500, 1000 times and compare the average run time
- Use the chrono timing measurement library (it comes standard in C++ 11, search for chrono time measurement C++) and repeat the above measurement.
- compare the two timing measurements.

References

[1] Timing measurement methods
https://www.geeksforgeeks.org/measure-execution-time-with-high-precision-in-c-c/
https://en.cppreference.com/w/cpp/chrono/c/clock